# HW 4 Core: Rotating Polarization with a Chain of Polarizers

Week 4 — Julia script submission

**Submission:** submit your Julia script on the course Moodle page.

## Core homework: a chain of polarizers

We start with light linearly polarized along $x$, so the electric field vector is $E_0 = (1, 0)$. We send it through $n + 1$ ideal polarizers whose transmission axes step evenly from just above the $x$-axis up to the $y$-axis ($90°$): for a given $n$ the axes are at

$$\theta_k = \frac{90° \, k}{n+1}, \qquad k = 1, 2, \ldots, n+1.$$

For $n = 8$ the axes are therefore at $10°, 20°, \ldots, 80°, 90°$. Each polarizer transmits only the component of the field along its axis, i.e. it applies the projection matrix

$$P(\theta) = \begin{pmatrix} \cos^2\theta & \cos\theta\sin\theta \\ \cos\theta\sin\theta & \sin^2\theta \end{pmatrix},$$

so after one polarizer the field becomes $E \leftarrow P(\theta_k)E$.

**Physics box: a polarizer is a projection**

An ideal polarizer with axis along the unit vector $u = (\cos\theta, \sin\theta)$ transmits only the component of the field along $u$: $E_{\text{after}} = (E \cdot u)u$. This is the linear projection $P(\theta)E$ with $P(\theta) = uu^T$, i.e. the $2\times2$ matrix above. The length of the outgoing field is $|E|\,|\cos(\phi-\theta)|$ when $E$ makes angle $\phi$ with the $x$-axis (Malus's law). A chain of many small rotations projects the field step by step; each step keeps a factor $\cos(\Delta\theta)$, so the total amplitude factor is $\cos(\Delta\theta)^n$ with $\Delta\theta = 90°/(n+1)$, which tends to $1$ as $n$ grows. In the limit the chain rotates the polarization by $90°$ with no loss. The product of the projections is not itself a rotation matrix (it is rank $1$), but on the input $(1, 0)$ it acts like the $90°$ rotation.

## Task

Write a Julia program that works for any given vector `npols` of polarizer-count values. You may assume that `npols` is already defined when the code runs (the check provides it, for example `npols = [1, 2, 4, 8, 16]`); do not define `npols` yourself. The program:

- loops over the values in `npols`;
- for each $n$ in `npols` starts from $E = (1, 0)$, loops $k = 1, \ldots, n+1$, computes $\theta_k$ in radians as `deg2rad(90*k/(n+1))`, builds $P(\theta_k)$, and updates $E \leftarrow P(\theta_k)E$;
- computes the outgoing amplitude

  $$\texttt{amplitude} = |E| = \sqrt{E_1^2 + E_2^2},$$

  and the intensity $\texttt{intensity} = \texttt{amplitude}^2$, the intensity of the radiation transmitted through the whole chain;
- appends that intensity to a vector `intensities`, one entry per $n$, so that when the program finishes, `intensities` holds the transmitted intensity for each of the values in `npols`, in the same order.

Do not define functions. Use only loops and `let` blocks. Round the final vector to six decimals with `intensities = round.(intensities; digits = 6)`. There is no need to print anything; the automatic check reads the vector `intensities` left by the program.

## Submission

Submit your Julia script on the course Moodle page. The script must run from a fresh Julia process with `npols` already defined by the check.

## Checking

With `npols = [1, 2, 4, 8, 16, 32, 64, 128, 256]` the expected vector is

$$\texttt{intensities} = [0.25, 0.421875, 0.605429, 0.759148, 0.864721, 0.927931, 0.962748, 0.981054, 0.990445],$$

rounded to six decimals. Check the smallest case by hand: for $n = 1$ the axes are $45^\circ$ and $90^\circ$, the first polarizer leaves $(1/2, 1/2)$, the second leaves $(0, 1/2)$, so amplitude $0.5$ and intensity $0.25$. Notice that the amplitude goes to unity: with enough steps the chain rotates the polarization by $90^\circ$ with almost no loss. Tell your friends how surprised you are.

## Optional extension (not for submission)

Plot the amplitude as a function of $n$ on a log scale for $n$ up to $10^4$, and estimate how many polarizers are needed to keep at least $99\%$ of the incident amplitude.